

PASSWORD HASHING & ENCRYPTION

Technical Reference Notes for SOC Analysts

1. PASSWORD HASHING

What is Hashing? A hash is a one-way mathematical transformation of data into a fixed-length string.

Core Concept: *Input* → *Hash Function* → *Fixed Length Output*

Examples of Hashing:

• **Password:** password123

MD5 Hash:

482c811da5d5b4bc6d497ffa98491e38

• **Password:** abcde1-23456f-ghijk789

bcrypt Hash:

2a12\$ks.tw-h2JnWGygZDS-Ff6WxuvMX1X.-kSFyjmu0E223-B4X04Kok8oe-win

Crucial Rule: No matter how long the input is, the output length remains strictly fixed.

Properties of Hashing:

- **One-way:** The transformation cannot be reversed easily to retrieve the plaintext.
- **Deterministic:** The exact same input will always produce the identical output hash.
- **Fixed output length:** Regardless of data size, the output size matches the algorithm standard.
- **Avalanche Effect:** A tiny change in the input yields a completely, unpredictably different output.

password123 vs password124 generate entirely distinct hashes.

Common Hash Algorithms:

Algorithm	Secure Today?	Notes
MD5	⊗ No	Very weak and mathematically vulnerable.
SHA1	⊗ No	Cryptographically broken.
SHA256	☑ Yes	Strong standard cryptographic hash.
SHA512	☑ Yes	Strong standard cryptographic hash.
bcrypt	☑ Very Strong	Slow by design; optimized explicitly for password hashing.
Argon2	☑ Very Strong	Modern standard, winner of Password Hashing Competition.

Why Hash Passwords?

Consider a standard web application database storing passwords in plaintext:

• **Username:** nitesh | **Password:** Nitesh@123

If the database is compromised, the attacker immediately gains full visibility of all user credentials. To mitigate this risk, websites store a hash representation instead:

• **Username:** nitesh | **Hash:** 5e884898da...

Even if the database leaks, the attacker must expend significant computational effort attempting to crack the hash.

2. THE BIG PROBLEM: RAINBOW TABLE ATTACKS (WITHOUT SALTING)

Hashing alone is **NOT** sufficient protection. Attackers deploy massive, precomputed look-up tables called **Rainbow Tables** mapping common plaintexts to known hashes:

Password Plaintext	Precomputed Hash
123456	e10adc3949ba...
password	5f4dcc3b5aa...
admin123	0192023a7bb...

Because hashing is deterministic, identical passwords across different users generate identical hashes:

- User 1 (password123) → Hash ABC
- User 2 (password123) → Hash ABC

An attacker instantly realizes that these users share the exact same password. If a stolen hash matches any entry inside a rainbow table, the password is instantly cracked. This flaw makes unsalted MD5/SHA1 implementations dangerous.

3. THE SOLUTION: SALTING & PEPPER

What is Salt? A salt is a unique, cryptographically random value appended to the plaintext password before it is passed to the hash function.

Salting Workflow:

1. User password: password123
2. Random salt generated: x7K9pL2
3. System computes: **$\text{Hash}(\text{password123} + \text{x7K9pL2})$** → Output: a83bd981fa...

If another user registers with the identical password, they are assigned a different random salt, resulting in a completely distinct hash. This neutralizes rainbow tables; attackers must brute-force each account individually, making large-scale cracking operations prohibitively expensive.

How Salting is Stored & Authenticated:

1. The database stores fields linearly: Username | Salt | Hash
2. When a user attempts login, the system retrieves the respective **Salt**.
3. The system appends this salt to the entered plaintext password.
4. The system hashes this combined string.
5. The output is compared to the stored hash; if they match, login succeeds.

Advanced Concept: Pepper

A **Pepper** is a secret value stored completely separate from the database (e.g., inside a hardened server configuration file). The system flow expands to: **$\text{Hash}(\text{Password} + \text{Salt} + \text{Pepper})$** . If the database leaks, the attacker remains unable to crack hashes without compromising the external application server config to retrieve the pepper.

4. HASHING VS. ENCRYPTION

Passwords should **NEVER** be encrypted. They must always be hashed and salted. The fundamental design requirements diverge significantly:

Feature	Hashing	Encryption
Reversible?	No	Yes
Used for Passwords?	Yes	No
Needs a Key?	No	Yes
Primary Purpose	Data Integrity & Verification	Data Confidentiality & Privacy
Output Length	Fixed length output	Variable length (scales with payload)
Example Implementation	User credential storage	HTTPS web traffic, VPN tunnels
Data Retrieval	Original data can never be derived	Original data cleartext is recoverable

5. WHAT IS ENCRYPTION?

Encryption is the process of translating readable data (**plaintext**) into an unreadable format (**ciphertext**) utilizing a cryptographic key, ensuring that only explicitly authorized parties possessing the matching key can decipher and read it.

- **Encryption Flow:** *Plaintext + Encryption Algorithm + Key → Ciphertext*
- **Decryption Flow:** *Ciphertext + Decryption Algorithm + Key → Plaintext*

Practical Example:

- Original Message: BankTransfer = ₹50,000
- Encrypted Ciphertext: A7F9C1B2E88DA91...

With the authorized cryptographic key, this ciphertext reverses perfectly back to the original text.

Common Use Cases for Encryption:

Encryption secures data-in-transit and data-at-rest across technical environments:

• HTTPS Web Traffic	• VPN Network Communication	• Full Disk Encryption (FDE)
• Secure Messaging Apps	• File-level Encryption	• Database Field-level Encryption

6. TYPES OF ENCRYPTION

A. Symmetric Encryption

Symmetric encryption relies on a **single shared secret key** used to execute both the encryption of plaintext and the decryption of ciphertext.

Algorithm	Operational Use Status
AES	The global standard for modern, high-speed bulk encryption.
DES	Legacy, deeply flawed, and completely insecure.
3DES	Legacy status; deprecated.
ChaCha20	Modern, highly secure stream cipher optimized for speed.

Example Uses: Full disk encryption, VPN tunnels, and local file storage protection.

The Core Problem: Key distribution. The single secret key must be shared securely between parties beforehand; if intercepted, total confidentiality is lost.

B. Asymmetric Encryption (Public Key Cryptography)

Asymmetric systems split architectural tasks across mathematically linked pairs: a **Public Key** (distributed openly) and a corresponding **Private Key** (kept strictly secret).

- The **Public Key** handles **Encryption** only.
- The **Private Key** handles **Decryption** only.

Algorithm	Primary Use Case
RSA	Standard asymmetric algorithm used extensively in SSL/TLS certificates.
ECC	Elliptic Curve Cryptography; high security with drastically smaller keys.
Diffie-Hellman	Secure key exchange mechanism over untrusted channels.

Example Uses: Secure HTTPS websites, initial SSL/TLS handshakes, and email encryption protocols (PGP).

HTTPS Connection Walkthrough:

1. Browser establishes connection and retrieves the Server's verified **Public Key**.
2. Browser generates a temporary symmetric session key and encrypts it using that **Public Key**.
3. Server receives the payload and decrypts it using its isolated, secure **Private Key**.
4. Both endpoints now securely share the symmetric session key, and high-speed encrypted transit begins.

SOC ANALYST PERSPECTIVE & INCIDENT RESPONSE

A comprehensive grasp of cryptographic architecture is mandatory when triaging alerts, analyzing malware, and managing active incident response workflows:

- **Data Breach Assessment:** During an exfiltration event, determining the cryptographic state of stolen data determines risk tiering. An exfiltrated *encrypted database* prevents visibility unless keys are compromised. An exfiltrated *un-salted password hash list* allows rapid offline cracking.
- **Credential Stuffing Trailing:** If a compromised third-party database relied on plain MD5 or unsalted SHA256, attackers will dump, reverse hashes via rainbow tables within minutes, and execute highly targeted credential stuffing automation campaigns across corporate assets (VPN, enterprise mail, internal portals).
- **Indicator of Compromise (IoC) Identification:** SOC analysts routinely spot weak cryptographic practices linked directly to security anomalies: login failures originating from distributed international infrastructure, rapid account takeover sequences, and impossible travel detection metrics. The ultimate root cause often traces back to deficient legacy hashing mechanisms within architecture.